

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 05 -

PROGRAMAÇÃO FUNCIONAL APLICADA A PIPELINES DE ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS.....	17

EMSE

O QUE VEM POR AÍ?

Pipelines de Machine Learning crescem rapidamente em complexidade e escrever código confiável torna-se um desafio cada vez maior. Esta aula apresenta a Programação Funcional como uma abordagem moderna para construir pipelines mais previsíveis, rastreáveis e fáceis de testar. São explorados conceitos essenciais como funções puras, imutabilidade e composição de funções, conectando teoria e prática de forma direta.

O(a) estudante aprenderá a estruturar pipelines como sequências claras de transformações, reduzindo erros silenciosos e aumentando a qualidade do código. Um exemplo prático em Python demonstra como esses princípios se aplicam a problemas reais de ciência de dados. Por fim, serão discutidos limites, trade-offs e a integração com outros paradigmas, te preparando para decisões técnicas mais maduras em projetos de Machine Learning.

HANDS ON

A crescente complexidade dos pipelines de Machine Learning impõe desafios à manutenção, confiabilidade e reprodutibilidade do código em projetos de ciência de dados. Abordagens baseadas exclusivamente em programação imperativa tendem a intensificar problemas como dependência excessiva de estado, efeitos colaterais difíceis de rastrear e maior dificuldade de depuração. Nesse contexto, a Programação Funcional surge como um paradigma capaz de tornar explícitas as transformações de dados ao longo do pipeline, reduzindo comportamentos inesperados e aumentando a previsibilidade do código.

O material apresenta os fundamentos da Programação Funcional, com destaque para funções puras, imutabilidade de dados e transparência referencial, estabelecendo um contraste direto com o paradigma imperativo tradicional. Esses conceitos são discutidos sob a perspectiva de sua aplicação em Machine Learning, enfatizando como a ausência de efeitos colaterais e a criação de novas versões dos dados contribuem para maior clareza e controle sobre o fluxo de processamento.

Em seguida, são explorados os principais benefícios da Programação Funcional em ciência de dados, como a melhoria da rastreabilidade das transformações, a facilitação da escrita de testes unitários e a separação mais clara entre as etapas do pipeline. Também são abordados o uso de funções de ordem superior e a composição de funções como mecanismos centrais para estruturar pipelines a partir de transformações simples, reutilizáveis e testáveis, consolidados por meio de um exemplo prático em Python envolvendo etapas de pré-processamento, treinamento e avaliação de modelos.

Por fim, são discutidos os limites da Programação Funcional e os trade-offs associados à sua adoção, especialmente no que se refere ao uso de dados imutáveis. A aula enfatiza a importância da integração equilibrada com outros paradigmas, em particular a Programação Orientada a Objetos, destacando o papel do Python como linguagem multiparadigma e preparando você para decisões arquiteturais mais conscientes em projetos reais de Machine Learning.

SAIBA MAIS

Nos últimos anos, os projetos de Machine Learning deixaram de ser scripts simples executados uma única vez e passaram a envolver pipelines complexos, compostos por diversas etapas encadeadas. Um pipeline típico pode incluir coleta de dados, limpeza, tratamento de valores faltantes, engenharia de atributos, normalização, treinamento de modelos, avaliação e monitoramento. À medida que essas etapas crescem em número e complexidade, o código também se torna mais difícil de manter, entender e confiar.

Em muitos projetos de ciência de dados, essas pipelines são implementadas utilizando um estilo imperativo de programação, no qual variáveis são criadas, modificadas e reutilizadas ao longo do código. Embora esse estilo seja intuitivo e amplamente utilizado, ele apresenta alguns problemas importantes quando aplicado a pipelines de ML.

Um dos principais problemas é a dependência de estado. Nesse modelo, o resultado de uma etapa depende não apenas de suas entradas explícitas, mas também do estado atual do programa — isto é, do valor que certas variáveis assumiram após execuções anteriores. Isso torna o comportamento do código mais difícil de prever, pois pequenas mudanças em uma parte do pipeline podem gerar efeitos inesperados em outras.

Outro ponto crítico são os efeitos colaterais. Um efeito colateral ocorre quando uma função altera algo fora de seu próprio escopo, como modificar uma variável global, alterar um objeto compartilhado ou gravar dados sem deixar isso explícito. Em pipelines de ML, efeitos colaterais podem fazer com que o mesmo código produza resultados diferentes dependendo da ordem de execução ou do contexto, o que dificulta a reprodutibilidade dos experimentos.

Além disso, códigos fortemente baseados em estado mutável tendem a ser mais difíceis de depurar e testar. Quando um erro ocorre, torna-se complicado identificar em qual etapa do pipeline o problema foi introduzido, já que várias transformações podem ter alterado os dados ao longo do caminho. Testes unitários também ficam mais complexos, pois testar uma função isoladamente pode exigir a reprodução de todo o estado do programa.

É nesse contexto que surge a Programação Funcional (FP – Functional Programming) como uma abordagem alternativa e complementar. A programação funcional propõe um modelo em que o programa é construído principalmente a partir de funções puras, que recebem dados como entrada e produzem novos dados como saída, sem modificar o estado externo. Em vez de alterar variáveis ao longo do código, cada etapa devolve um novo resultado, tornando o fluxo de dados mais explícito.

Ao adotar princípios da programação funcional em pipelines de Machine Learning, ganhamos maior confiabilidade, pois cada etapa tem um comportamento previsível; maior previsibilidade, já que a saída depende apenas da entrada; e maior reprodutibilidade, aspecto fundamental em projetos científicos e industriais de ML. Isso facilita tanto o desenvolvimento quanto a manutenção dos pipelines, além de tornar o código mais legível, testável e robusto ao longo do tempo.

A seguir, veremos como esses conceitos funcionam na prática e como podem ser aplicados diretamente em pipelines de dados e modelos utilizando Python.

Fundamentos da programação funcional

A Programação Funcional é um paradigma de programação que trata a computação como a avaliação de funções matemáticas, evitando mudanças de estado e efeitos colaterais observáveis. Diferentemente de paradigmas baseados em comandos e instruções sequenciais, a abordagem funcional busca descrever o que deve ser computado, não como o estado do programa deve ser modificado ao longo do tempo. Essa característica torna a programação funcional especialmente adequada para contextos em que previsibilidade, rastreabilidade e reprodutibilidade são fundamentais, como em pipelines de Machine Learning.

Em projetos de ciência de dados, é comum que pipelines sejam compostos por diversas transformações encadeadas sobre um mesmo conjunto de dados. Quando essas transformações são implementadas de forma imperativa, utilizando variáveis mutáveis e alterações sucessivas de estado, o código passa a carregar uma complexidade implícita difícil de controlar. A programação funcional surge como uma resposta direta a esse problema ao impor regras claras sobre como dados podem ser manipulados.

Funções puras

O conceito de função pura está no centro da programação funcional. Uma função é considerada pura quando satisfaz duas propriedades fundamentais. A primeira é o determinismo: para um mesmo conjunto de entradas, a função sempre produz a mesma saída. Isso significa que o resultado da função não depende de variáveis globais, do estado do sistema, do tempo de execução ou de qualquer outro fator externo. A segunda propriedade é a ausência de efeitos colaterais. Uma função pura não altera variáveis fora de seu escopo, não modifica objetos compartilhados, não realiza operações de entrada ou saída e não provoca mudanças observáveis no ambiente do programa.

Do ponto de vista matemático, uma função pura pode ser entendida como um mapeamento entre conjuntos: dados valores de entrada pertencentes a um domínio, a função retorna um valor pertencente a um contradomínio, sem interferir em qualquer outro elemento do sistema. Essa visão é particularmente poderosa quando aplicada a pipelines de ML, pois cada etapa do pipeline passa a representar uma transformação bem definida dos dados.

Considere, por exemplo, uma função responsável por normalizar uma variável numérica. Se essa função apenas recebe um vetor de valores e retorna um novo vetor normalizado, sem alterar o vetor original, ela é pura. Em contrapartida, se a função modifica o vetor original in-place ou depende de parâmetros externos definidos em outro ponto do código, ela deixa de ser pura. Em pipelines complexos, pequenas impurezas como essas se acumulam, tornando o comportamento do sistema difícil de prever.

A adoção sistemática de funções puras traz benefícios diretos para testes e validação. Como não dependem de contexto externo, essas funções podem ser testadas de forma isolada, com entradas controladas e saídas esperadas claramente definidas. Isso é particularmente relevante em Machine Learning, onde erros sutis em etapas de pré-processamento podem comprometer todo o modelo sem serem facilmente detectados.

Imutabilidade de dados

Outro princípio fundamental da programação funcional é a imutabilidade. Em um contexto funcional, dados não devem ser modificados após sua criação. Em vez disso, cada transformação gera uma nova versão dos dados, preservando as versões anteriores. Essa abordagem contrasta fortemente com o estilo imperativo tradicional, no qual estruturas de dados são frequentemente alteradas in-place ao longo do programa.

A imutabilidade reduz significativamente a complexidade cognitiva envolvida na compreensão de um pipeline. Quando dados são mutáveis, o estado atual de uma variável depende de toda a sequência de operações anteriores, o que exige que a pessoa desenvolvedora rastreie mentalmente cada modificação realizada. Em pipelines de ML, onde os dados passam por diversas transformações, isso rapidamente se torna impraticável.

Ao trabalhar com dados imutáveis, cada etapa do pipeline passa a ter uma entrada e uma saída bem definidas. Isso facilita a depuração, pois é possível inspecionar os resultados intermediários sem o risco de que eles sejam alterados por etapas posteriores. Além disso, a imutabilidade favorece a reprodutibilidade dos experimentos, já que os dados originais permanecem preservados ao longo de todo o processo.

É importante reconhecer que a imutabilidade pode implicar um maior consumo de memória, especialmente quando grandes volumes de dados são copiados. No entanto, em muitos cenários de ciência de dados, esse custo é compensado pela clareza, segurança e confiabilidade do código. Além disso, diversas bibliotecas modernas utilizam estratégias internas de otimização para minimizar esse overhead, como compartilhamento estrutural e avaliação preguiçosa.

Em pipelines de Machine Learning, a imutabilidade permite inspecionar resultados intermediários sem o risco de que etapas futuras modifiquem dados passados, facilitando auditoria e depuração.

Transparência referencial

A transparência referencial é uma consequência direta do uso de funções puras e dados imutáveis. Um trecho de código é referencialmente transparente quando pode ser substituído pelo seu valor resultante sem alterar o comportamento do programa. Em termos práticos, isso significa que uma chamada de função pode ser mentalmente “avaliada” e substituída por seu resultado, sem a necessidade de considerar efeitos colaterais ou dependências externas.

Esse conceito tem implicações profundas para a forma como programas são entendidos e mantidos. Em um pipeline referencialmente transparente, cada etapa pode ser analisada de maneira independente, o que facilita a leitura, a validação e a refatoração do código. Para pipelines de ML, isso significa que cada transformação de dados pode ser compreendida como um bloco autônomo, cuja lógica é explícita e verificável.

Além disso, a transparência referencial abre espaço para técnicas avançadas de otimização e paralelização, uma vez que chamadas de função independentes podem ser reordenadas ou executadas em paralelo sem risco de alterar o resultado final. Embora nem sempre explorado diretamente em Python, esse princípio é amplamente utilizado em frameworks e sistemas distribuídos de processamento de dados.

Contraste com o paradigma imperativo

Para compreender plenamente o valor da programação funcional, é fundamental contrastá-la com o paradigma imperativo. No modelo imperativo, o programa é descrito como uma sequência de instruções que alteram o estado do sistema. Variáveis são criadas, atualizadas e reutilizadas ao longo do tempo e o significado de uma variável depende de todo o histórico de operações que a afetaram.

Na programação funcional, por outro lado, o foco está no retorno de novos valores, não na modificação de valores existentes. Cada função expressa uma transformação clara e o fluxo de dados é explícito. Essa diferença de abordagem tem impacto direto na qualidade do código, especialmente em pipelines de Machine Learning, onde a clareza e a previsibilidade são cruciais.

É importante destacar que a programação funcional não substitui completamente o paradigma imperativo, especialmente em linguagens multiparadigma como Python. No entanto, a adoção consciente de seus princípios — funções puras, imutabilidade e transparência referencial — pode elevar significativamente o nível de robustez, testabilidade e confiabilidade dos pipelines de ML. Um(a) desenvolvedor(a) experiente sabe combinar esses paradigmas de forma estratégica, escolhendo a abordagem mais adequada para cada problema.

A adoção de princípios da Programação Funcional em Machine Learning não é motivada apenas por preferências estilísticas, mas por necessidades práticas que surgem à medida que pipelines se tornam mais longos, complexos e críticos para decisões de negócio ou científicas. Em contextos reais, um pipeline de ML raramente é uma sequência curta de operações; ele envolve múltiplas transformações de dados, validações, experimentos, ajustes de hiperparâmetros e reexecuções frequentes. Nesse cenário, a forma como o código é estruturado tem impacto direto na confiabilidade dos resultados.

A seguir, discutimos os principais benefícios da Programação Funcional aplicada a pipelines de ML, sempre ilustrando com exemplos concretos.

Um dos maiores desafios em pipelines de Machine Learning é entender como os dados foram transformados ao longo do processo. Em abordagens imperativas, é comum que os dados sejam modificados repetidamente in-place, dificultando a reconstrução do caminho percorrido até o resultado final.

Na Programação Funcional, cada transformação é representada por uma função explícita, que recebe dados de entrada e devolve novos dados como saída. Isso cria um histórico lógico claro das transformações aplicadas.

A previsibilidade é uma consequência direta do uso de funções puras e dados imutáveis. Quando uma função não depende de estado externo e não produz efeitos colaterais, seu comportamento torna-se fácil de antecipar e verificar.

Testes unitários são fundamentais em Machine Learning, especialmente para validar etapas de pré-processamento e engenharia de atributos. A Programação Funcional facilita esses testes porque funções puras não exigem preparação de contexto, mocks complexos ou estado prévio.

A Programação Funcional incentiva a decomposição do pipeline em funções pequenas, coesas e especializadas, cada uma responsável por uma única transformação. Isso melhora a legibilidade e promove o princípio da responsabilidade única.

Apesar dos benefícios, a Programação Funcional não é isenta de trade-offs. O principal deles é o custo de memória, já que a criação de novas versões dos dados pode consumir mais recursos do que modificações in-place.

No entanto, em muitos cenários de Machine Learning, esse custo é compensado pela:

- Maior clareza do código.
- Menor probabilidade de erros silenciosos.
- Facilidade de depuração e auditoria.
- Confiabilidade dos experimentos.

Além disso, diversas bibliotecas modernas utilizam otimizações internas para mitigar esse impacto, como avaliação preguiçosa e compartilhamento estrutural.

Funções de ordem superior e composição de pipelines

À medida que pipelines de Machine Learning crescem em complexidade, torna-se cada vez mais importante construir um código que seja flexível, reutilizável e fácil de reorganizar. A Programação Funcional oferece mecanismos poderosos para isso por meio das funções de ordem superior e da composição de funções. Esses conceitos permitem tratar funções como entidades de primeira classe, capazes de serem passadas como argumentos, retornadas por outras funções e combinadas para formar pipelines expressivos e previsíveis.

Em Machine Learning, essa abordagem favorece a criação de pipelines modulares, nos quais cada etapa representa uma transformação clara dos dados, e a lógica de orquestração do pipeline é separada da lógica das transformações individuais.

Uma função de ordem superior é uma função que recebe outra função como argumento, retorna uma função como resultado ou ambos. Em Python, funções são

objetos de primeira classe, o que significa que podem ser atribuídas a variáveis, passadas como parâmetros e retornadas como valores.

Esse conceito é central na programação funcional, pois permite abstrair padrões de comportamento e reutilizar lógica de forma elegante.

Além de receber funções como argumentos, uma função de ordem superior também pode retornar outra função. Esse padrão é útil quando queremos gerar transformações parametrizadas de forma controlada.

Funções de ordem superior permitem que pipelines sejam parametrizados por comportamento, não apenas por dados. Isso significa que o pipeline pode ser adaptado a diferentes cenários sem modificar sua implementação interna.

A composição de funções consiste em combinar funções simples para formar transformações mais complexas. Em vez de escrever uma função grande e monolítica, a programação funcional incentiva a construção de pequenas funções coesas que podem ser encadeadas.

Matematicamente, se temos duas funções f e g , a composição $f(g(x))$ representa a aplicação sequencial dessas transformações.

Em pipelines de Machine Learning, a composição de funções permite modelar o processamento de dados como uma cadeia de transformações explícitas. Cada função representa uma etapa conceitual do pipeline.

Cada função:

- Tem uma responsabilidade clara.
- Pode ser testada isoladamente.
- Pode ser reutilizada em outros pipelines.

Essa abordagem facilita tanto a manutenção quanto a evolução do sistema, pois alterações em uma etapa não exigem reescrita do pipeline inteiro.

Limites da programação funcional e integração com outros paradigmas

A Programação Funcional oferece um conjunto poderoso de princípios para lidar com complexidade, previsibilidade e confiabilidade em pipelines de Machine Learning. No entanto, é essencial compreender que esses princípios não constituem uma solução universal. Nenhum paradigma de programação é capaz de atender, de forma ideal, a todos os tipos de problemas. O uso consciente da Programação Funcional exige não apenas o domínio de seus conceitos, mas também a compreensão clara de seus limites e de seus pontos de integração com outras abordagens.

Um dos primeiros aspectos a considerar é que a Programação Funcional não é sempre a melhor escolha. Ela se mostra especialmente eficaz quando o problema pode ser modelado como uma sequência de transformações de dados, com entradas e saídas bem definidas. Entretanto, em situações onde o sistema depende fortemente de interação contínua com o ambiente, controle de fluxo complexo ou operações que exigem resposta imediata a eventos externos, a tentativa de manter uma abordagem estritamente funcional pode introduzir complexidade desnecessária. Nesses contextos, a clareza e a simplicidade de abordagens imperativas ou orientadas a objetos tendem a ser mais adequadas.

Outro ponto fundamental diz respeito à necessidade de estado mutável em alguns cenários. Embora a imutabilidade seja um dos pilares da Programação Funcional, o estado mutável não deve ser encarado como algo intrinsecamente negativo. Em Machine Learning, por exemplo, existem processos naturalmente iterativos, como treinamento de modelos, atualização de parâmetros, monitoramento de métricas ao longo do tempo e controle de ciclos de execução. O problema central não é a existência de estado, mas sim o estado implícito, compartilhado e não controlado. A Programação Funcional contribui ao ensinar que o estado, quando necessário, deve ser explícito, localizado e facilmente rastreável, mesmo que não seja completamente eliminado.

A integração com bibliotecas e frameworks existentes também impõe limites práticos ao uso da Programação Funcional. Grande parte do ecossistema de ciência de dados e Machine Learning em Python foi construída sob forte influência de paradigmas imperativos e orientados a objetos. Esses frameworks frequentemente

utilizam objetos com estado interno, métodos que modificam esse estado e APIs desenhadas para encapsular comportamento e dados conjuntamente. Tentar contornar ou abstrair completamente essas características pode gerar camadas artificiais que aumentam a complexidade do código sem ganhos reais. Uma abordagem mais madura consiste em integrar princípios funcionais onde eles trazem benefícios claros, sem ignorar a forma como essas bibliotecas foram concebidas.

Nesse ponto, torna-se relevante uma comparação breve entre Programação Funcional e Programação Orientada a Objetos. A Programação Funcional busca reduzir a complexidade por meio da eliminação de efeitos colaterais e da separação clara entre dados e transformações. Já a Programação Orientada a Objetos lida com a complexidade por meio do encapsulamento de estado e comportamento em entidades bem definidas. Em sistemas de Machine Learning, é comum que a Programação Funcional seja mais adequada para etapas de pré-processamento, transformação e composição de pipelines, enquanto a Programação Orientada a Objetos se destaca na representação de modelos, serviços, componentes reutilizáveis e integração com frameworks. Esses paradigmas não competem entre si; ao contrário, se complementam quando usados de forma consciente.

O Python, por sua vez, é uma linguagem explicitamente multiparadigma, o que a torna particularmente adequada para esse tipo de integração. Ela permite que funções puras, composição funcional, classes, objetos e controle imperativo coexistam de maneira natural dentro do mesmo projeto. Essa flexibilidade oferece ao indivíduo desenvolvedor a liberdade de escolher o paradigma mais apropriado para cada parte do sistema, desde que essa escolha seja guiada por critérios técnicos e não por preferência ideológica.

Em síntese, a Programação Funcional deve ser vista como uma ferramenta poderosa dentro de um conjunto mais amplo de abordagens disponíveis à pessoa desenvolvedora de Machine Learning. Seu uso eficaz depende da capacidade de reconhecer seus limites, compreender quando outros paradigmas são mais adequados e integrar essas abordagens de forma coerente. Essa visão crítica e equilibrada é essencial para a construção de sistemas de ML robustos, manuteníveis e alinhados às demandas do mundo real.

O QUE VOCÊ VIU NESTA AULA?

O desenvolvimento de pipelines de Machine Learning tem se tornado progressivamente mais complexo à medida que modelos, volumes de dados e exigências de reprodutibilidade aumentam. Em cenários reais, pipelines não se limitam a etapas isoladas de pré-processamento ou treinamento, mas envolvem múltiplas transformações encadeadas, validações, experimentação contínua e integração com sistemas externos.

Nesse contexto, abordagens tradicionais baseadas exclusivamente em programação imperativa frequentemente apresentam limitações relacionadas à previsibilidade do código, à dependência excessiva de estado e à dificuldade de rastrear transformações aplicadas aos dados. Esses fatores motivam a adoção de paradigmas que ofereçam maior controle conceitual sobre o fluxo de processamento, entre os quais se destaca a Programação Funcional.

A Programação Funcional propõe um modelo de desenvolvimento centrado na aplicação de funções como transformações explícitas de dados, reduzindo a dependência de estado mutável e minimizando efeitos colaterais. Ao contrário do paradigma imperativo, no qual variáveis são alteradas ao longo do tempo e o significado do código depende da sequência de instruções executadas, a abordagem funcional enfatiza a clareza do fluxo de dados e a previsibilidade do comportamento do sistema.

Um benefício relevante é a facilidade de escrita de testes unitários. Como funções puras não dependem de contexto externo, elas podem ser testadas de forma direta e determinística, com entradas controladas e saídas esperadas claramente definidas. Essa característica é especialmente importante em Machine Learning, onde erros em etapas iniciais do pipeline podem se propagar silenciosamente até fases posteriores do processo. A Programação Funcional também favorece uma separação mais clara entre as etapas do pipeline, permitindo que cada transformação seja desenvolvida, testada e evoluída de forma independente.

Apesar dessas vantagens, a Programação Funcional também apresenta trade-offs. O principal deles está relacionado ao custo de memória e, em alguns casos, ao desempenho, devido à criação de múltiplas versões dos dados. Esses custos, no

entanto, devem ser analisados à luz dos ganhos em clareza, segurança e confiabilidade do código. Em muitos projetos de Machine Learning, especialmente aqueles voltados à pesquisa, experimentação ou sistemas críticos, esses ganhos superam as desvantagens associadas.

Apesar de suas vantagens, a Programação Funcional não deve ser aplicada de forma dogmática. Existem cenários em que o uso de estado mutável é natural ou necessário, como em processos iterativos de treinamento, monitoramento de métricas ou integração com sistemas externos. O problema central não é a existência de estado, mas a presença de estado implícito, compartilhado e difícil de rastrear. Uma abordagem madura consiste em isolar e controlar o estado quando ele é inevitável, combinando princípios funcionais com outras abordagens de forma consciente.

O Python, como linguagem multiparadigma, oferece suporte natural a essa integração. A possibilidade de utilizar funções puras, composição funcional, classes, objetos e estruturas imperativas no mesmo projeto permite ao desenvolvedor ou desenvolvedora escolher a abordagem mais adequada para cada parte do sistema. Essa flexibilidade exige, no entanto, critérios técnicos claros e uma compreensão profunda das implicações de cada paradigma.

Em síntese, os conteúdos abordados fornecem uma base conceitual e prática sólida para a utilização da Programação Funcional em pipelines de Machine Learning. Ao compreender seus fundamentos, benefícios, limitações e possibilidades de integração com outros paradigmas, você passa a dispor de ferramentas conceituais para projetar sistemas mais robustos, previsíveis e manuteníveis. Essa visão equilibrada é essencial para lidar com a complexidade crescente dos projetos de Machine Learning, tanto no contexto acadêmico quanto no profissional.

REFERÊNCIAS

ABELSON, H.; SUSSMAN, G. J. **Structure and interpretation of computer programs**. 2. ed. Cambridge: MIT Press, 1996.

CHIOUSANO, P.; BJARNASON, R. **Functional programming in Scala**. Shelter Island: Manning Publications, 2015.

MERTZ, D. **Functional programming in Python**. Sebastopol: O'Reilly Media, 2015.

MILEWSKI, B. **Category theory for programmers**. [S. l.]: [s. n.], 2019.

RAMALHO, L. **Fluent Python**. 2. ed. Sebastopol: O'Reilly Media, 2022.

PALAVRAS-CHAVE

Composição. Previsibilidade. Imutabilidade.

EMENDADO



POSTECH